

GEORISK-SPARK

Real-Time Geospatial Supply Chain Disruption and Route Intelligence

PROJECT OVERVIEW AND SYSTEM GUIDE

GeoRisk-Spark is a Big Data Analytics project concept for detecting possible vessel exposure to marine hazards, measuring shipping-corridor congestion, estimating delays, and comparing safer routes. It combines event streaming, distributed processing, geospatial indexing, cloud storage, analytics, and an interactive dashboard.

One-sentence idea: convert moving vessels and changing hazards into H3 cell events, then use streaming analytics to identify risk and make route decisions easier to inspect.

Why it matters

Supply chains rely on vessels moving through ports, chokepoints, and sea lanes. Storms and congestion can create delays, but alerts and position feeds are often examined in separate tools. Traditional point-in-polygon checks become costly when applied repeatedly to large, fast GPS streams. GeoRisk-Spark uses H3 hexagonal cells as an indexing layer so a vessel point and hazard coverage can be compared through compact cell identifiers before any finer geometry check.

What the finished demonstration will show

- Synthetic vessel telemetry and time-bounded marine hazard alerts flowing into a stream.
- H3-based candidate risk alerts and 15-minute unique-vessel density by cell.
- Historical event storage and SQL analysis in an AWS data lake.
- An ETA-delay estimate and a risk-aware alternative path on a small shipping graph.
- A map dashboard combining live positions, hazards, density, alerts, estimates, and detours.

Important status

The repository currently contains scaffold only: module placeholders, JSON Schema contracts, configuration guidance, and project documentation. The features in this guide describe the intended completed system; they are not implemented or deployed yet.

1. Product features

The project turns several related data products into one operational picture. Each item below is a planned feature for the semester demonstration.

Feature	What it does	User-facing result
Vessel event replay	Produces repeatable positions with vessel ID, time, latitude/longitude, speed, and heading.	A controllable stream that can be replayed for development and demos.
Weather alert intake	Represents storms, high winds, or high-wave zones as GeoJSON polygons with severity and validity times.	Hazards appear and expire according to their event data.
Spatial risk detection	Maps points and hazard coverage to H3 cells and matches vessel cells to active hazard cells.	Flagged vessels with the matching alert, severity, and cell ID.
Corridor density	Groups events into 15-minute event-time windows and counts distinct vessels per H3 cell.	Hex-level density metric for congested corridors.
Delay prediction	Uses spatial density and hazard severity as features in a Spark MLlib regression model.	Estimated delay in hours for an active route, labeled as a synthetic demonstration.
Dynamic route comparison	Scores graph edges by travel cost and risk; recalculates a path when an edge exceeds a chosen threshold.	Baseline path beside a lower-risk alternative, with cost and risk shown.
Interactive dashboard	Combines vessel positions, hazards, H3 heat, alerts, delay estimates, and path overlays.	A single view for exploring current simulated conditions and historical patterns.

Example user journey

A synthetic vessel position arrives near an active high-wave alert. The processor maps the point to an H3 cell, matches it to the alert's covered cells, and emits a risk record. The same cell contributes to corridor density for its 15-minute window. If the route uses a lane with high density or hazard risk, the route service compares the baseline with an alternative path. The dashboard shows the vessel, alert, density, estimated delay, and route comparison together.

The risk result is a decision-support signal for a project demo. H3 matches are approximate and must not be presented as exact polygon containment or real-world safety advice.

2. System architecture and data flow

The planned deployment uses managed AWS services for event ingestion, Spark processing, lake storage, and SQL analysis. The same event contracts should also support a small local development mode.

SYNTHETIC SOURCES	INGEST	SPATIAL STREAM	LAKE + CATALOG	PRODUCTS
Vessel GPS Weather polygons	Amazon Kinesis two streams	AWS Glue + Spark H3 risk + density	S3 Parquet Glue Catalog	Athena, MLlib routes, dashboard

Layer	Responsibilities	Main outputs
Ingestion	Generate and validate events; publish vessel telemetry and weather alerts to separate Kinesis Data Streams.	Versioned JSON event records.
Stream processing	AWS Glue runs Spark Structured Streaming. Decode events, assign H3 cells, maintain active hazard coverage by validity time, identify candidate risk, and compute event-time density.	Risk events and unique-vessel density rows.
Lake storage	Write raw and processed data to separate S3 prefixes in Parquet; store checkpoints for stream recovery; publish schema metadata in Glue Data Catalog.	Date-partitioned historical data.
Analytics and routing	Run Athena queries and Spark MLlib training/evaluation. Score a compact port and sea-lane graph and calculate a risk-aware alternative route.	Historical summaries, delay estimates, route alternatives.
Visualization	Streamlit reads current outputs and renders positions, alerts, density, estimates, and paths on an interactive map.	Interactive project demonstration.

S3 layout direction

Keep raw and processed data separate, and partition primarily by event date. Example prefixes:

``raw/vessel_telemetry/event_date=YYYY-MM-DD/'`, ``raw/weather_alerts/event_date=YYYY-MM-DD/'`, ``curated/risk_events/event_date=YYYY-MM-DD/'`, and ``curated/density_metrics/event_date=YYYY-MM-DD/'`. Store H3 resolution and cell ID as columns rather than making a separate S3 partition for every cell.

3. H3 spatial indexing and risk logic

H3 is a hierarchical discrete global grid. A latitude/longitude point maps to one cell at a chosen resolution; a polygon maps to a set of cells. This gives the streaming pipeline a compact spatial key for grouping and candidate matching.

Operation	Planned behavior	Why it matters
Point indexing	Map every accepted vessel position to one H3 cell, starting with resolution 7; allow resolution 8 for a documented comparison.	Stable spatial key for grouping, joins, and maps.
Hazard coverage	Convert each active GeoJSON Polygon or MultiPolygon to H3 cells with an explicit containment rule.	Transforms a complex shape into a set that can be compared to vessel cells.
Candidate match	Match vessel H3 ID against active hazard H3 coverage; attach alert ID, type, severity, and validity.	Fast first-pass spatial screening and explainable alert output.
Density	Window by event time, deduplicate vessel ID per H3 cell/window, then count.	Measures unique vessel concentration instead of raw message volume.

Boundary and coordinate rules

- Vessel events carry latitude and longitude as separate values. GeoJSON polygon positions use longitude then latitude.
- The standard H3 polygon-to-cells operation uses cell-center containment; cells that only partially overlap a polygon can be omitted. The H3 library also documents experimental overlap containment.
- For the risk screen, choose an overlap-aware cover where available so boundary cells are not silently missed. The candidate set can be refined with exact point-in-polygon checks when needed.
- Keep event time, hazard valid-from/valid-until, H3 resolution, and selected coverage rule in the processing contract and logs.

Risk output

A risk record links a vessel event to its H3 cell and any active hazard cells that match. Include an event ID, event time, vessel ID, coordinates (or a protected reference), H3 index, resolution, risk flag, and a list of matching alert IDs and severity values. Keep the original alert validity in the match so dashboards can explain why a flag exists.

H3 turns geometry work into a fast candidate lookup. It does not make a polygon test exact by itself. Boundary coverage mode and any geometry-refinement step must be visible in the project results.

4. Data contracts and analytics

The repository includes version 1 JSON Schemas for the four shared record types. These contracts keep the ingestion, processing, storage, and dashboard layers aligned.

Record	Core fields
Vessel telemetry	schema_version, event_id, event_time, vessel_id, latitude, longitude, speed_knots, heading_deg
Weather alert	schema_version, event_id, alert_id, event_time, valid_from, valid_until, hazard_type, severity, GeoJSON geometry
Risk event	event_time, vessel_id, h3_index, h3_resolution, risk_alert, matching_hazards
Density metric	window_start, window_end, h3_index, h3_resolution, unique_vessel_count

Congestion metrics

Use 15-minute event-time windows. For each H3 cell and window, deduplicate repeated telemetry from the same vessel before counting unique vessel IDs. The resulting density table can be queried over a time range to reveal busy corridors, compare affected cells, and support route scoring.

Delay prediction

- Train a Spark MLlib Gradient-Boosted Tree regression model on synthetic historical examples.
- Candidate features: route baseline travel time, recent H3 vessel density, hazard severity/exposure, vessel speed, and route distance.
- Define the synthetic delay label with a documented formula so the demo is reproducible. It is not evidence of real-world predictive accuracy.
- Use a time-based train/evaluation split. Report MAE and RMSE and compare them with a mean-delay baseline.

Route optimization

Represent ports and chokepoints as graph nodes and shipping lanes as edges. An edge carries estimated sailing time and a current risk score derived from hazard severity and congestion. When an edge exceeds a configured threshold, recalculate the path with a risk penalty or mark that edge unavailable. Show baseline travel cost, alternative travel cost, and risk difference side by side. The semester graph can be small; GraphFrames can be evaluated if it fits the Spark runtime, with a standard shortest-path implementation as a fallback.

5. Dashboard and user experience

The dashboard is the presentation layer for the pipeline. It should help a viewer answer: where are vessels now, which cells are exposed or congested, what delay is expected, and how does the route change?

View	Contents	Interaction
Live map	Vessel points, active storm polygons, H3 risk cells, flagged alerts, and detour lines.	Pan/zoom; toggle layers; select a vessel or alert for details.
Risk summary	Active alert count, vessels exposed, severity distribution, and latest event time.	Filter by alert type, severity, and time range.
Corridor density	Hexagonal heat layer and per-cell unique vessel count for each 15-minute window.	Select a cell and compare adjacent windows.
Delay view	Estimated delay by vessel or route with feature context and a baseline.	Compare prediction with the synthetic ground-truth label in demo mode.
Route comparison	Baseline route and risk-aware alternative with lane status and travel/risk cost.	Change threshold or hazard scenario and recalculate.
Data freshness	Latest stream event, output watermark, and processing lag where available.	Make stale or incomplete data visible rather than showing a misleading live state.

Suggested build

- Use Streamlit for the application shell and controls, with PyDeck/deck.gl for map layers and H3 extrusions.
- Use local fixtures during early development, then use an Athena query adapter for cloud-backed views.
- Show units, timestamps, H3 resolution, alert validity, and whether a value is simulated.
- Keep a clear no-data state for missing streams, expired alerts, and empty query results.

Demonstration sequence

Start synthetic replay; show fresh telemetry on the map; introduce an alert polygon; show H3 candidate cells and an emitted risk event; inspect density for the current 15-minute window; show a delay estimate and its baseline; exceed a lane risk threshold; compare the rerouted path; finish with a queryable historical summary.

6. AWS operation, reliability, and cost

The planned stack uses managed services to keep infrastructure work contained. The project should run cloud compute only for bounded development sessions and demonstrations, then stop or remove resources that are not needed.

Service	Purpose	Operational note
Amazon Kinesis Data Streams	Receive vessel telemetry and weather alerts.	Two logical streams; size for demo throughput and set a documented retention period.
AWS Glue Streaming	Run Spark Structured Streaming for validation, H3 processing, alert matching, and density.	Checkpoint progress in S3. Glue streaming jobs are billed while running; run only for planned tests/demos.
Amazon S3	Store raw events, checkpoints, and processed Parquet datasets.	Use separate prefixes, event-date partitions, encryption, and lifecycle/cleanup rules.
AWS Glue Data Catalog	Catalog table schemas and S3 locations.	Keep table schemas aligned with versioned contracts.
Amazon Athena	Run SQL over data in S3 through cataloged tables.	Use Parquet and date filters to limit data scanned; configure query-result location.
CloudWatch	Observe job logs, errors, throughput, and lag.	Review logs during each run; avoid unnecessary long retention for a student demo.

Security and data handling

- Generate synthetic vessel records; do not collect personal or sensitive live vessel data for the course demonstration.
- Use least-privilege IAM roles and a dedicated project bucket. Keep credentials in a local AWS profile or SSO session, not in source files.
- Separate raw and curated data and set explicit retention and teardown procedures.
- Use UTC for timestamps and record event time separately from processing time.
- Check current regional availability, permissions, and prices before provisioning; free-tier or student credit coverage may vary.

Failure behavior

Malformed records should be counted and quarantined rather than silently treated as valid. Streaming checkpoints allow recovery from the last committed progress. Duplicate events should be controlled with stable IDs and bounded deduplication. Late events should follow a documented watermark/window policy. If the dashboard cannot retrieve current results, show freshness and an explicit unavailable state.

7. Build phases, success criteria, and current state

A suggested 12-week sequence keeps the complete path visible while features are added progressively.

Phase	Focus	Exit evidence
Weeks 1-3	Contracts, local generator, cloud account checks, and a small working stream design.	Valid event examples and decisions for H3 resolution, alert coverage, and output fields.
Weeks 4-6	Kinesis ingestion and AWS Glue Spark streaming for spatial alerts and window density.	Short replay reaches an inspectable S3 risk and density output with checkpoint and expiry behavior.
Weeks 7-9	S3 data lake layout, Glue Catalog, Athena SQL, MLlib delay baseline.	Historical queries match expected counts; model reports holdout metrics against baseline.
Weeks 10-12	Risk-aware routes, dashboard, end-to-end integration, evaluation, and presentation.	Repeatable demo shows all planned capabilities and states the limits of synthetic data and H3 matching.

Project acceptance

- Valid telemetry inside active hazard coverage raises a candidate risk alert; clearly outside events do not.
- Expired hazards stop matching, and duplicate messages do not inflate unique-vessel density.
- 15-minute cell-level density can be queried from S3 with Athena.
- Delay estimates are evaluated against a mean baseline on a held-out time period.
- A route above the risk threshold produces an explainable alternative path.
- The dashboard shows spatial and analytic outputs with timestamps, units, freshness, and simulation labels.

Current code status

This PDF describes the intended full project. The GitHub repository currently contains project scaffolding only: placeholder modules, schemas, and documentation. No Kinesis streams, Glue job, S3 lake, Athena tables, model, route optimizer, or dashboard have been implemented or deployed.

Technical references

AWS Glue streaming ETL: <https://docs.aws.amazon.com/glue/latest/dg/add-job-streaming.html>

AWS Glue streaming concepts and sources: <https://docs.aws.amazon.com/glue/latest/dg/streaming-chapter.html>

Athena SQL over S3 and Glue Catalog: <https://docs.aws.amazon.com/athena/latest/ug/using-athena-sql.html>

H3 region functions and polygon coverage modes: <https://h3geo.org/docs/api/regions/>

H3 index representation: <https://h3geo.org/docs/core-library/h3indexing/>